

ISOVALENT  
now part of cisco

# netkit: Specializing Linux Packet Delivery for Container Networks

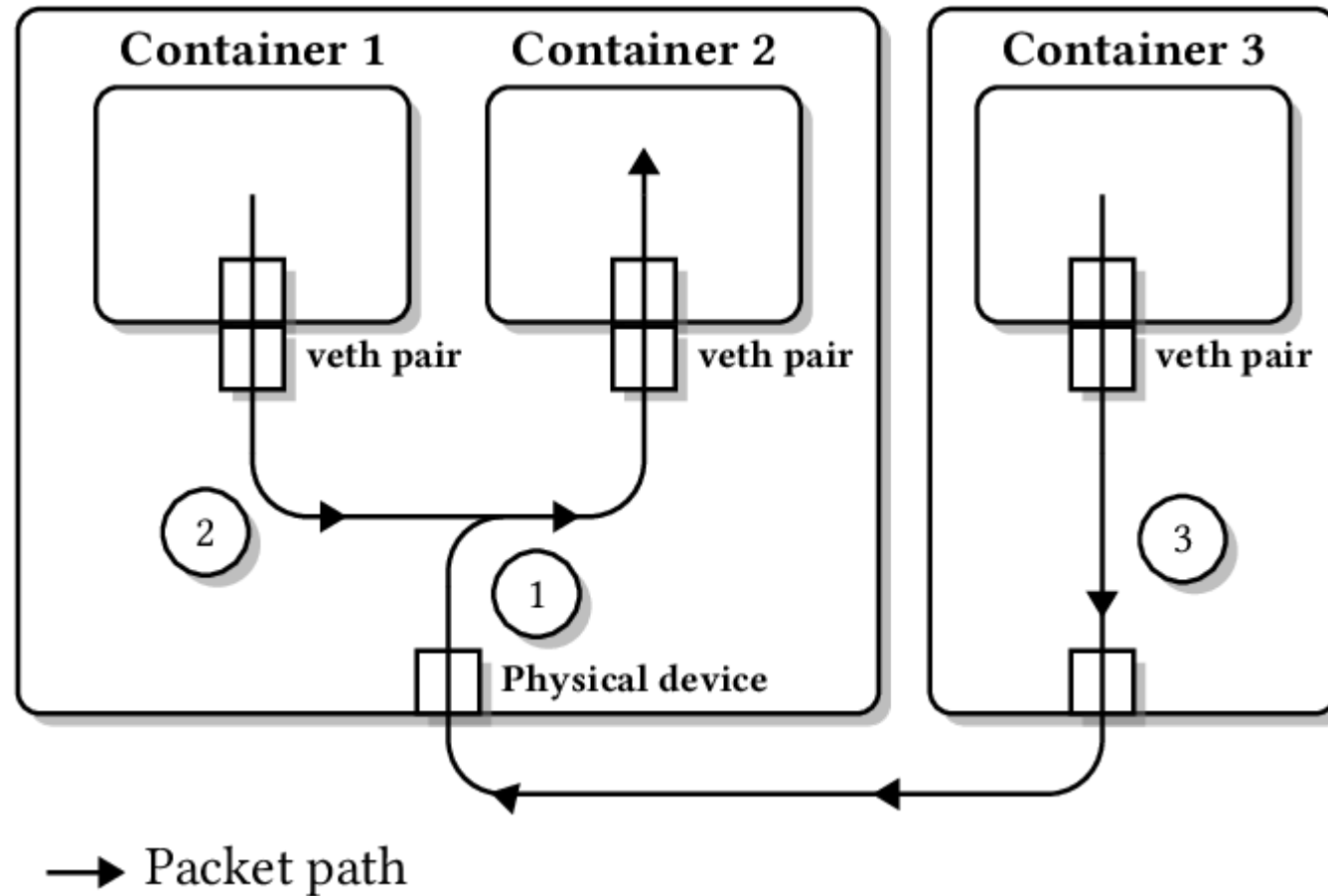
---

Daniel Borkmann, Isovalent at Cisco  
Paul Chaignon, Isovalent at Cisco

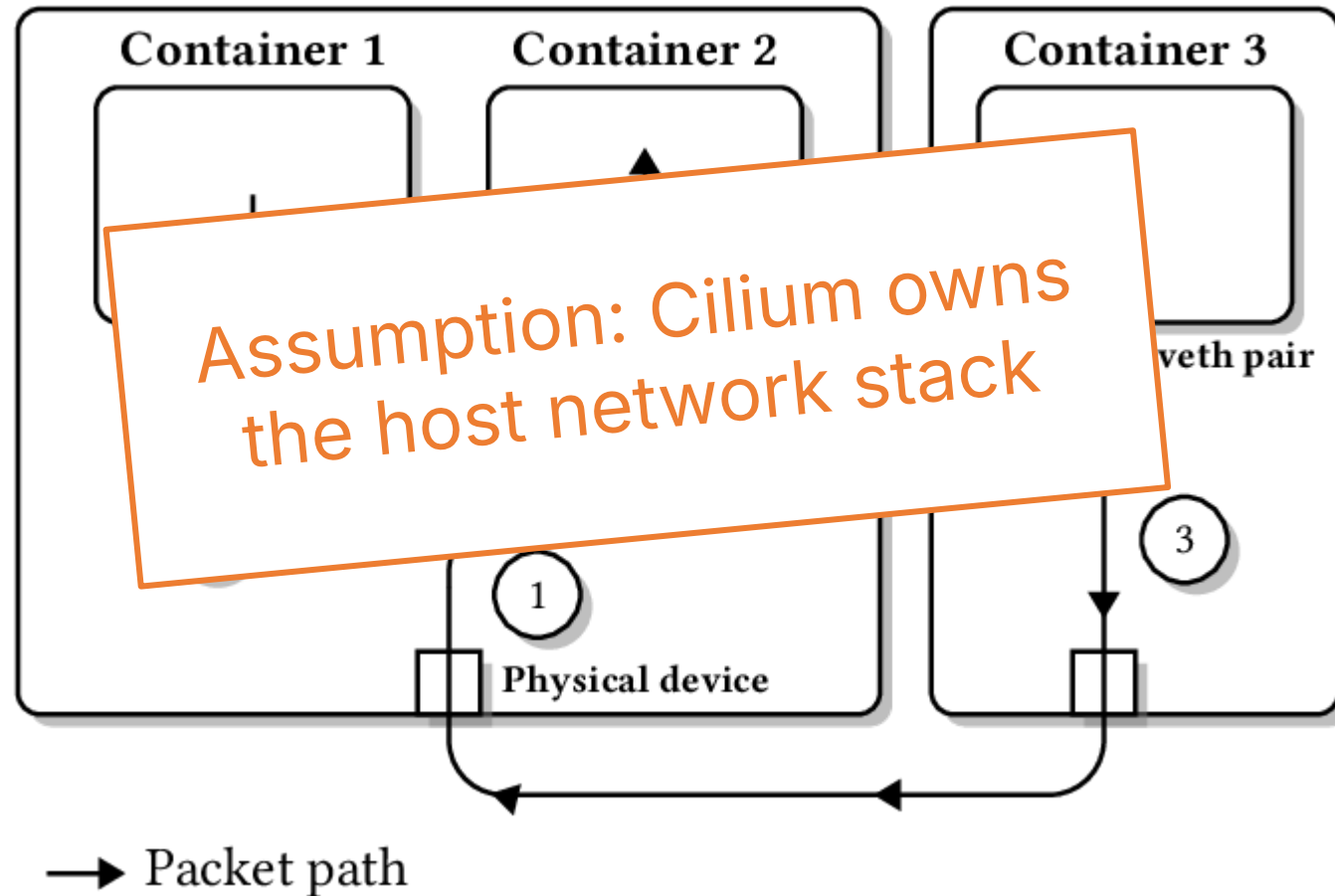


29-09-2026

# Introduction: Container Networking



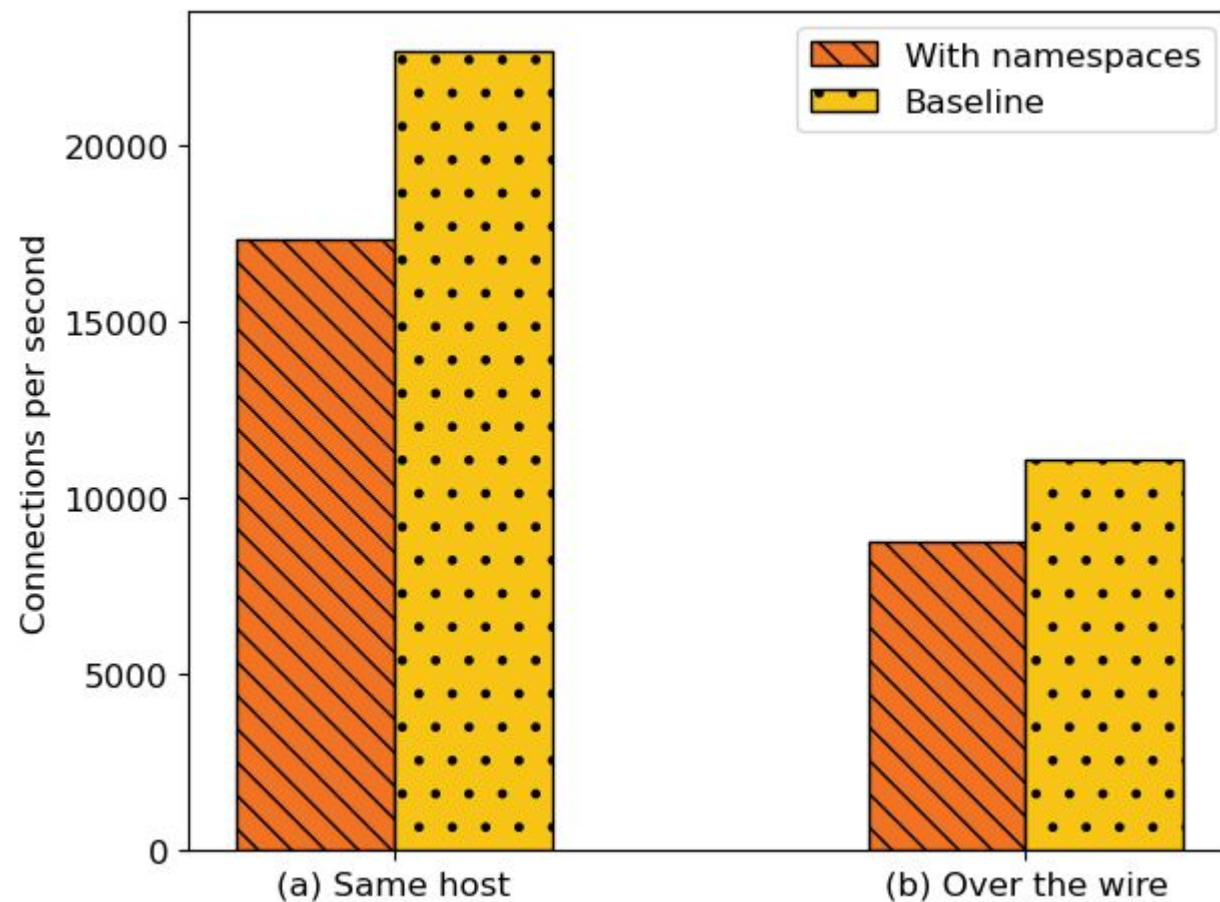
# Introduction: Container Networking



# Motivation: Gap to Baseline

- netperf TCP\_CRR
- Over loopback and wire
- With and without namespaces

Unexpected. Network namespaces are logical rather than physical boundaries.



# Related Works: BPF sockmap



## HybridMesh: A Hardware-software Hybrid Accelerating Service Mesh Ingr

Myoungsung You Jaehyun Nam  
University of Seoul Dankook University  
Taeyune Park Seung  
Chonnam National University

### Abstract

Service meshes have become essential for enabling microservice communication in cloud environments. Despite their benefits, they impose significant network overhead. We identify that the ingress gateway, the main entry point for external traffic, has emerged as a major performance bottleneck. This overhead stems from two key limitations of current ingress gateways: (1) CPU-intensive traffic analysis and (2) a complex packet forwarding path. Our analysis shows that these inefficiencies can reduce network throughput by up to 4x while substantially increasing CPU utilization. In response, we propose *HybridMesh*, a hardware-software hybrid ingress gateway that leverages a SmartNIC for high-performance traffic analysis and efficient traffic routing. This process is augmented by a lightweight CPU-side proxy that provides various traffic management features not suitable for SmartNIC execution. Evaluations show that *HybridMesh* outperforms existing ingress gateways, achieving a 4.4x increase in HTTP throughput. Furthermore, compared to software-only and hardware-only designs, our hybrid design delivers 2.3x higher cost-effectiveness and 2.8x better power-efficiency.

### 1 Introduction

Service meshes [51] are becoming the de facto standard communication infrastructure for implementing large-scale microservices on cloud environments. A recent survey revealed that 47% of organizations deploy their applications on service meshes [2]. Operating as an intermediate layer, service meshes decouple communication logic between microservices, such as peer discovery or load-balancing, from application logic. This separation simplifies the application development process and enhances the manageability of microservices. Service meshes, while central to facilitating microservice communication, often introduce substantial network overhead due to their inherent complex design. A notable example of such overhead is attributed to sidecars [34]. Sidecars reverse proxies deployed alongside each application to

## Canal Mesh: A Cloud-Scale Sidecar-Free Multi-Tenant Service Mesh Architecture

Enge Song<sup>1\*</sup>, Yang Song<sup>1\*</sup>, Chengyun Lu<sup>1</sup>, Tian Pan<sup>1,2</sup>, Shaokai Zhang<sup>1</sup>, Jianyuan Lu<sup>1</sup>, Jiangui Zhao<sup>1</sup>, Xining Wang<sup>1</sup>, Xiaomin Wu<sup>1</sup>, Minglan Gao<sup>1</sup>, Zongquan Li<sup>1</sup>, Ziyang Fang<sup>1</sup>, Biao Lyu<sup>1</sup>, Pengyu Zhang<sup>1</sup>, Rong Wen<sup>1</sup>, Li Yi<sup>1</sup>, Zhigang Zong<sup>1,3</sup>, Shunmin Zhu<sup>1,4</sup>  
<sup>1</sup>Alibaba Cloud <sup>2</sup>Zhejiang University <sup>3</sup>Tsinghua University  
alibaba\_cloud\_network@alibaba-inc.com

### ABSTRACT

In recent years, service mesh frameworks have gained significant popularity in building microservice-based applications. A key component of these frameworks is a proxy in each K8s pod, named sidecar, which handles inter-pod traffic. Our empirical measurement reveals that such per-pod sidecars cause numerous problems, including intrusion into the user pod, excessive resource occupation, significant overhead in managing many sidecars, and performance degradation caused by passing traffic through the sidecar.

In this paper, we introduce *Canal Mesh*, a cloud-scale sidecar-free multi-tenant service mesh architecture. Canal decouples service mesh functions from the user cluster and deploys a centralized mesh gateway in the public cloud to handle these functions, thus reducing user intrusion and orchestration overhead. Through service consolidation and multi-tenancy, the infra costs of service mesh are also reduced. To address the rising issues due to cloud-based deployment, such as service availability, tenant isolation, noisy neighbor, service elasticity, and additional infra costs, we leverage techniques including hierarchical failure recovery, shuffle sharding, rapid intervention, precise scaling, cloud infra reuse and resource aggregation, etc. Our evaluation shows that Canal Mesh's performance, resource consumption, and control plane overhead are significantly better than Istio and Ambient. We also share experiences from years of deploying Istio and Canal in production.

### CCS CONCEPTS

• Networks → Cloud computing, Middle boxes / network appliances; Network performance analysis.

### KEYWORDS

microservice, service mesh, sidecar, public cloud, multi-tenancy, service consolidation, centralized mesh gateway

### ACM Reference Format:

Enge Song, Yang Song, Chengyun Lu, Tian Pan, Shaokai Zhang, Jianyuan Lu, Jiangui Zhao, Xining Wang, Xiaomin Wu, Minglan Gao, Zongquan Li, Ziyang Fang, Biao Lyu, Pengyu Zhang, Rong Wen, Li Yi, Zhigang Zong, Shunmin Zhu. 2024. Canal Mesh: A Cloud-Scale Sidecar-Free Multi-Tenant Service Mesh Architecture. In *Proceedings of ACM SIGCOMM 2024 Conference (ACM SIGCOMM '24)*. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3651890.3672221>

\*Both authors contributed equally to this work. \*Co-corresponding authors.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).  
ACM SIGCOMM '24, August 4-8, 2024, Sydney, NSW, Australia  
© 2024 Copyright held by the owner(s). Publication rights licensed to ACM.  
ACM ISBN 978-0-497-96141-1/24/08  
<https://doi.org/10.1145/3651890.3672221>

Fang, Biao Lyu, Pengyu Zhang, Rong Wen, Li Yi, Zhigang Zong, Shunmin Zhu. 2024. Canal Mesh: A Cloud-Scale Sidecar-Free Multi-Tenant Service Mesh Architecture. In *Proceedings of ACM SIGCOMM 2024 Conference (ACM SIGCOMM '24)*. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3651890.3672221>

### 1 INTRODUCTION

Service mesh has emerged as an infrastructure that enables service-to-service communication over networks [41, 42, 47, 51, 55, 68]. Major cloud providers, such as AWS [20], Azure [21], GCP [28], Alibaba [19], have launched service mesh-based products that simplify building and managing microservice-based applications. A key component in many service mesh frameworks, such as Istio [30] and Linkerd [32], is a proxy named sidecar. The sidecar takes charge of the network traffic management for a pod, handling tasks such as policy-based routing, percentage-based traffic splitting, rate limiting, etc. Decoupling network functions into a sidecar allows flexible traffic management without changing the user's service logic.

After deploying Istio on Alibaba Cloud to serve customers for over four years [19], our production data indicates that such a per-pod sidecar deployment causes the following problems:

- **Intrusion:** The sidecar sits inside the user pod and runs alongside the user app. Its presence in the user pod may cause potential security and stability issues. For example, memory leakage in a sidecar could crash the user app within the same pod.
- **Throughput and latency:** The throughput could degrade by 3x-7x and the latency could increase by 3x-7x after adopting the sidecar [61]. This is because each request needs to be redirected to the sidecar, which introduces extra processing steps [61], significantly increasing the performance overhead [2-4, 11, 61].
- **Resource occupation:** When loaded with complex network and security configurations, the sidecar consumes substantial CPU and memory, which are purchased by users originally for running their apps. Moreover, maintaining optimal performance of user services requires that the sidecar's CPU utilization stay unsaturated (e.g., < 45%), which exacerbates the resource issue.
- **Orchestration:** Since each pod has one sidecar, and a cloud service could have  $O(100k)$  pods, the overhead of orchestrating configurations for these sidecars is extremely large [4].

To address these problems, Cilium [23] and SPRIGHT [61] use eBPF [25] to reduce packet processing latency and CPU consumption. Despite their advantages, the eBPF-based solutions have limited programmability, making it challenging to support flexible Layer 7 (L7) processing, such as HTTP. According to our investigation with customers, 80%-95% of service mesh users need L7 functions. Therefore, both Cilium and SPRIGHT are insufficient to meet our customers' demands. Ambient [38] is a pioneering

## Extracting the Server from Serverless Computing! Enhance eBPF-based Event-driven, Shared-memory Processing

Leslie Monis, Ziteng Zeng, Ian-chin Wang, K. K. Ramakrishnan  
University of California, Riverside, CA  
Netherlands, ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3544216.3544239>

### 1 INTRODUCTION

efficient, low-cost compute carrier, existing solutions, epitomized as Knative, include heavy-lifts lack dataplane optimization function chains that support event-driven capabilities for building serverless applications.

performance, response time, shared memory, probability of the dataplane, and serialization. Packet Filter (eBPF) leverages event-driven mechanisms to support strictly load-balancing with 10x less latency. eBPF benefits upstreams, allowing RIGHT achieves high latency, low CPU usage, and

ing. • Net-

ishman, High-ACM, sidem.

Poor dataplane performance for function chaining. Modern cloud-native architectures decompose the monolithic application into container-per-Pod<sup>1</sup> is the most common model used by Kubernetes for running a function instance.

Serverless computing has grown in popularity because users have to only develop their applications while depending on a cloud service provider to be responsible for managing the underlying operating system and hardware infrastructure. The typical costs borne by the user are only for processing incoming requests. This event-driven consumption of resources is attractive for cloud users, especially when their demand is intermittent. It does, however, place the burden on the cloud service provider to provide adequate resources on-demand and ensure the quality of service requirements are met. In many cases, serverless frameworks are prodigal in their resource consumption. They provide the needed functionality by loosely coupling serverless functions and middleware components that run as a separate container and/or pod<sup>1</sup>. This can be extremely resource-intensive, especially when deployed in a limited capacity environment, e.g., edge cloud [37]. There are still a number of shortcomings to be overcome for building a high-performance, resource-efficient, and responsive serverless cloud. Some contributors to this overhead are the following.

**Use of heavyweight serverless components.** In a serverless environment, each function pod has a dedicated sidecar proxy, distinct from its application container. Sidecar proxies help build an inter-function service mesh layer with extensive functionality support, e.g., metrics collection and buffering, facilitating serverless networking and orchestration. However, the existing serverless is heavyweight since it is continuously running and incurs excessive overheads, including 2 data copies, 2 context switches, and 2 interrupts (see §2) for a single request. Moreover, since most serverless frameworks primarily focus on HTTP/REST API [6, 25, 47], e.g., IoT (Internet-of-Things) with MQTT [19, 73], CoAP [38]. The current design runs protocol adaptation as an individual component in resource-limited edge clouds or when handling infrequent workloads (e.g., IoT). Instead, going a step further and invoking code for execution on a completely event-driven basis without using an individual component can result in substantial resource savings.

# Related Works: BPF sockmap

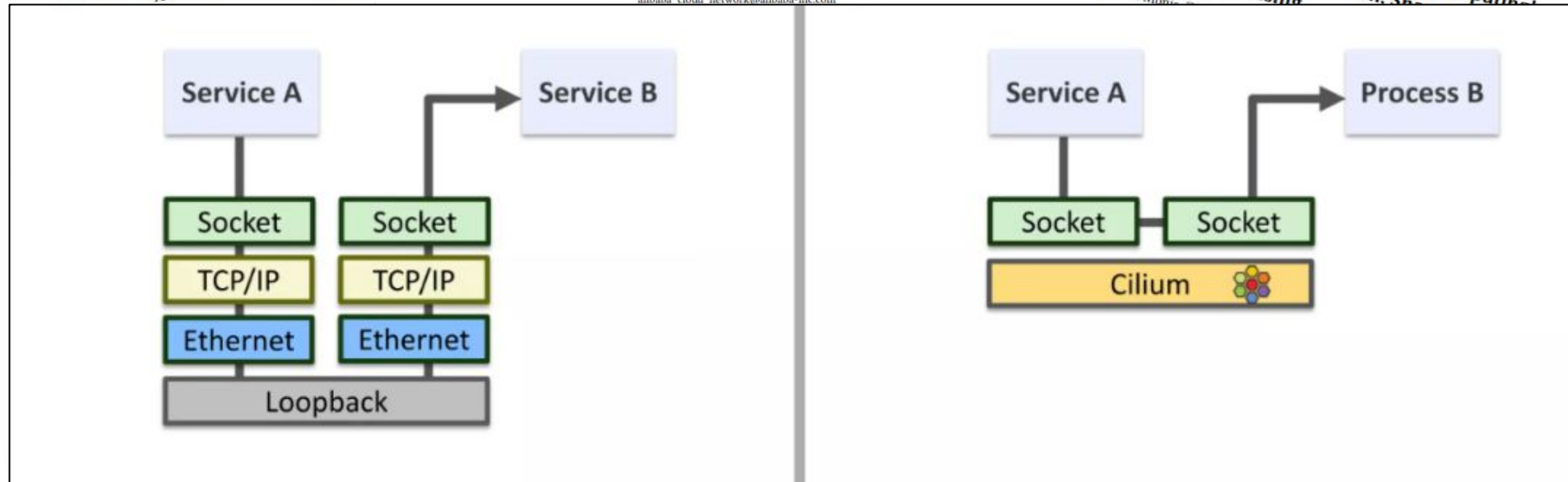


HybridMesh: A Hardware-software Hybrid  
Accelerating Service Mesh Ingr

## Canal Mesh: A Cloud-Scale Sidecar-Free Multi-Tenant Service Mesh Architecture

Enge Song<sup>†\*</sup>, Yang Song<sup>†\*</sup>, Chengyun Lu<sup>†</sup>, Tian Pan<sup>†□</sup>, Shaokai Zhang<sup>†</sup>, Jianyuan Lu<sup>†</sup>,  
Jiangui Zhao<sup>†</sup>, Xining Wang<sup>†</sup>, Xiaomin Wu<sup>†</sup>, Minglan Gao<sup>†</sup>, Zongquan Li<sup>†</sup>, Ziyang Fang<sup>†</sup>,  
Biao Lyu<sup>†</sup>, Pengyu Zhang<sup>†</sup>, Rong Wen<sup>†</sup>, Li Yi<sup>†</sup>, Zhigang Zong<sup>†□</sup>, Shunmin Zhu<sup>††□</sup>  
<sup>†</sup>Alibaba Cloud <sup>‡</sup>Zhejiang University <sup>□</sup>Tsinghua University  
alibaba\_cloud\_network@alibaba-inc.com

Extracting the Server from Serverless Computing  
Performance eBPF-based Event-driven, Shared  
Processing



### 1 Introduction

Service meshes [51] are becoming the de facto standard communication infrastructure for implementing large-scale microservices on cloud environments. A recent survey revealed that 47% of organizations deploy their applications on service meshes [2]. Operating as an intermediate layer, service meshes decouple communication logic from application services, such as peer discovery or load-balancing, from application logic. This separation simplifies the application development process and enhances the manageability of microservices. Service meshes, while central to facilitating microservice communication, often introduce substantial network overhead due to their inherent complex design. A notable example of such overhead is attributed to sidecars [34]. Sidecars are reverse proxies deployed alongside each application to

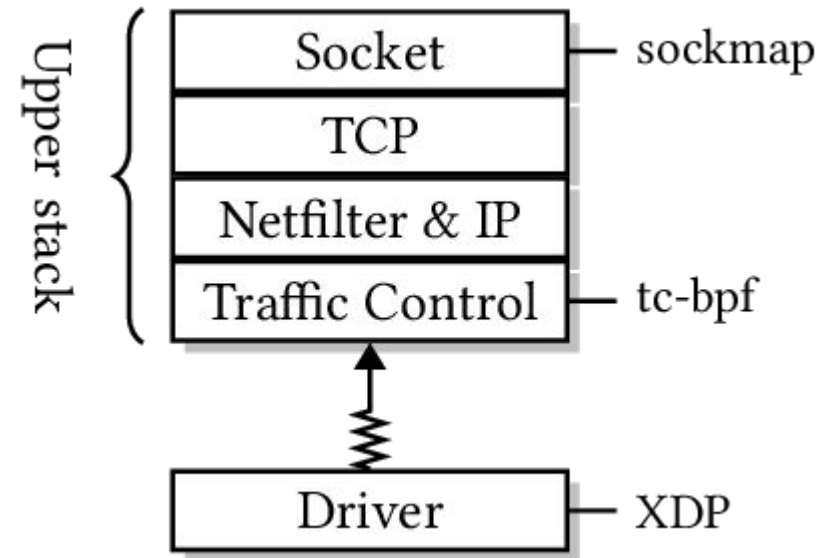
\*Both authors contributed equally to this work. †Co-corresponding authors.  
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.  
ACM SIGCOMM '24, August 4–8, 2024, Sydney, NSW, Australia  
© 2024 Copyright held by the owner/authors. Publication rights licensed to ACM.  
ACM ISBN 978-0-497-4614-1/24/08  
<https://doi.org/10.1145/3633399.3672221>

service could have  $O(100k)$  pods, the overhead of orchestrating configurations for these sidecars is extremely large [4].  
To address these problems, Cilium [23] and SPRIGHT [61] use eBPF [25] to reduce packet processing latency and CPU consumption. Despite their advantages, the eBPF-based solutions have limited programmability, making it challenging to support flexible Layer 7 (L7) processing, such as HTTP. According to our investigation with customers, 80%-95% of service mesh users need L7 functions. Therefore, both Cilium and SPRIGHT are insufficient to meet our customers' demands. Ambient [38] is a pioneering

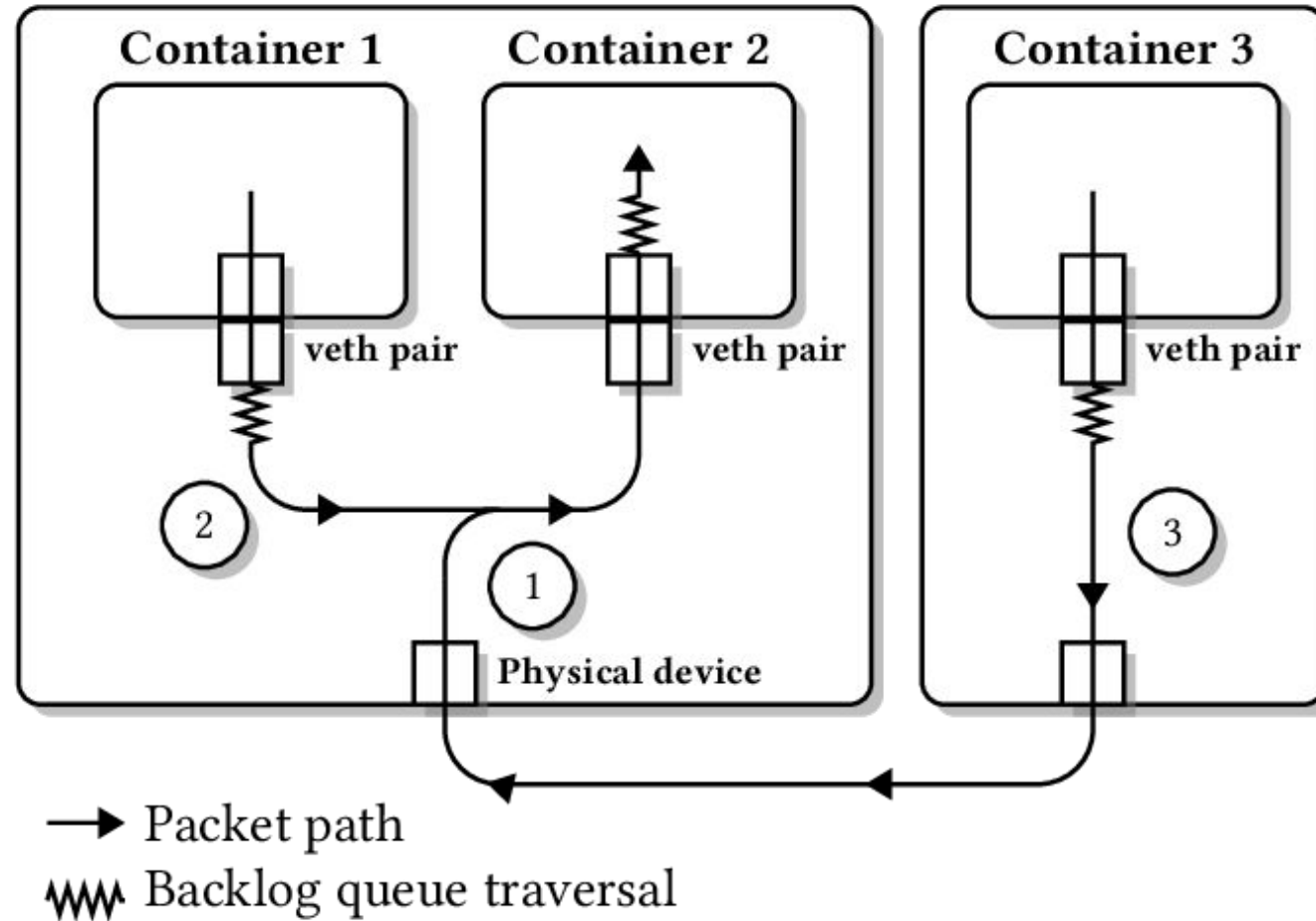
request. Moreover, since most serverless environments are designed for lightweight protocols (e.g., IoT (Internet-of-Things) is required for specialized use cases, resulting in substantial resource consumption. Having such a heavyweight design may overload serverless environments, especially in resource-limited edge clouds or when handling infrequent workloads (e.g., IoT). Instead, going a step further and invoking code for execution on a completely event-driven basis without using an individual component can result in substantial resource savings.  
Poor dataplane performance for function chaining. Modern cloud-native architectures decompose the monolithic application into containers-per-Pod<sup>1</sup> is the most common model used by Kubernetes for running a function instance.

# Background: Backlog Queue

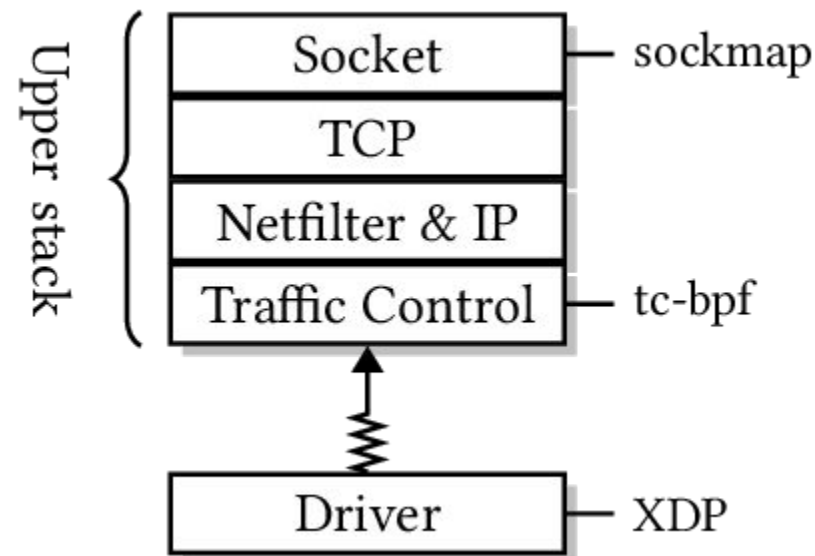
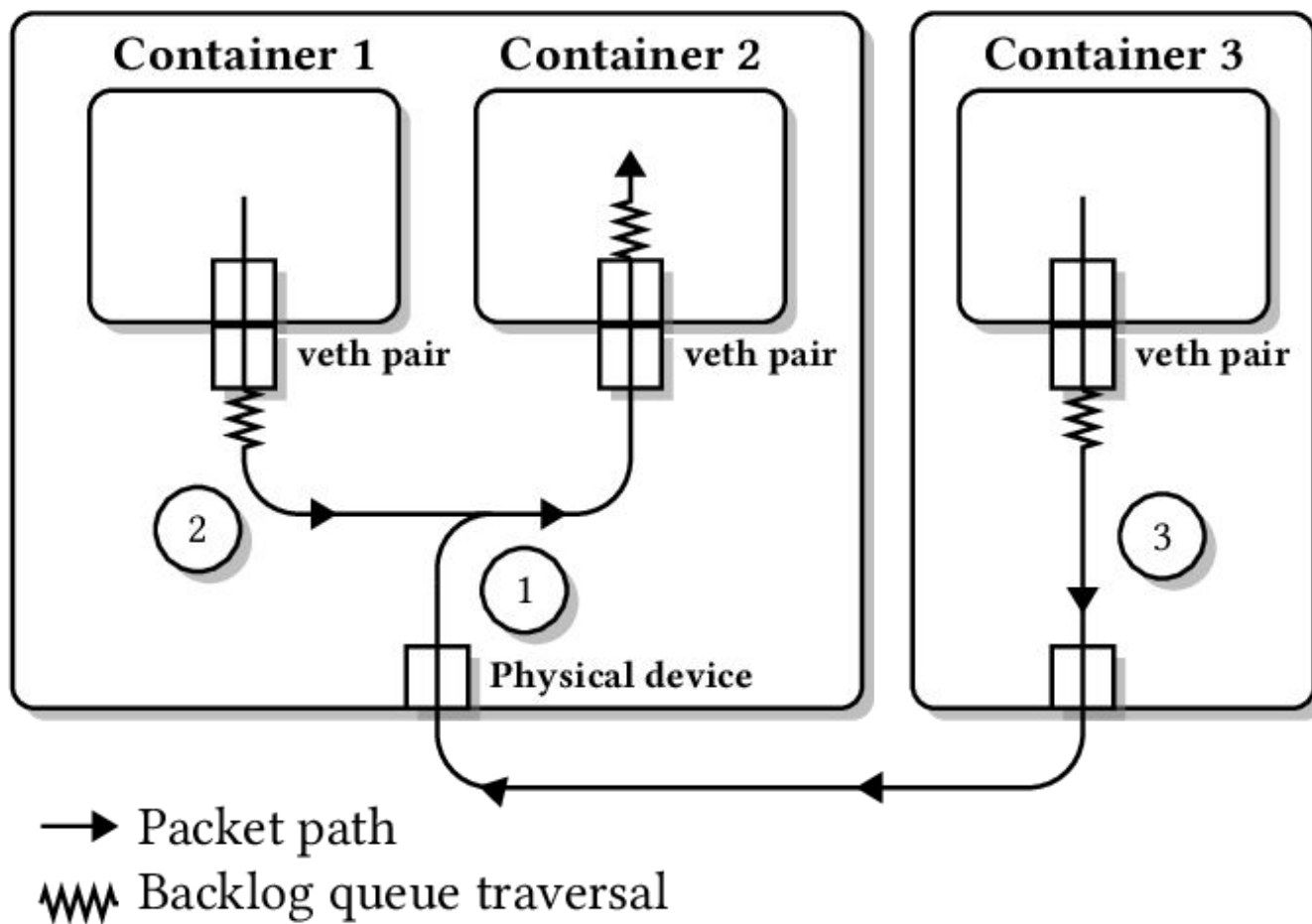
- Per-CPU backlog queue between network driver and upper stack.
- Backlog queues used to scale across cores (ex. RPS). Not the default.
- Always used for Virtual Ethernet devices (veth).
  - Not always desirable!



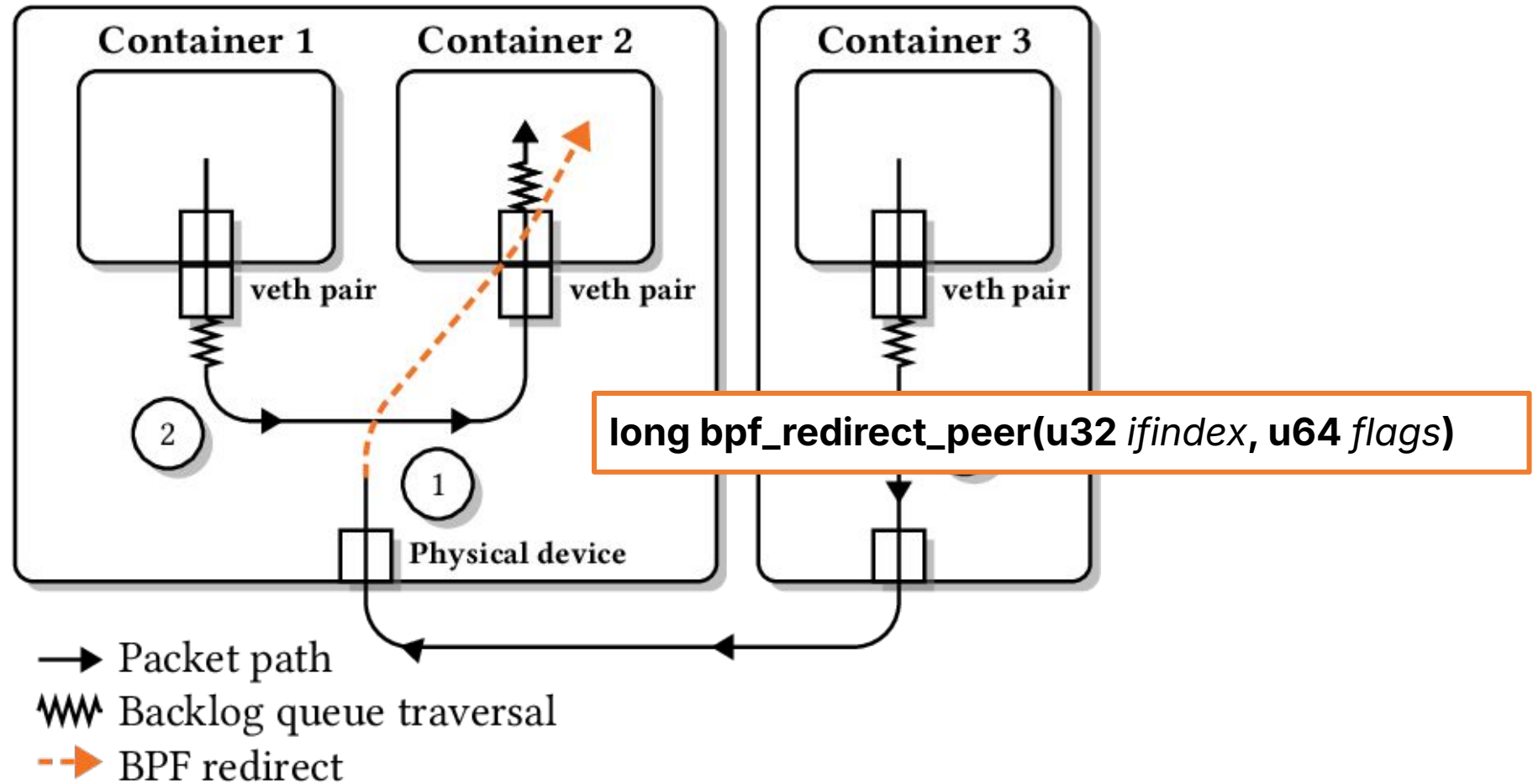
# Background: Backlog Queue



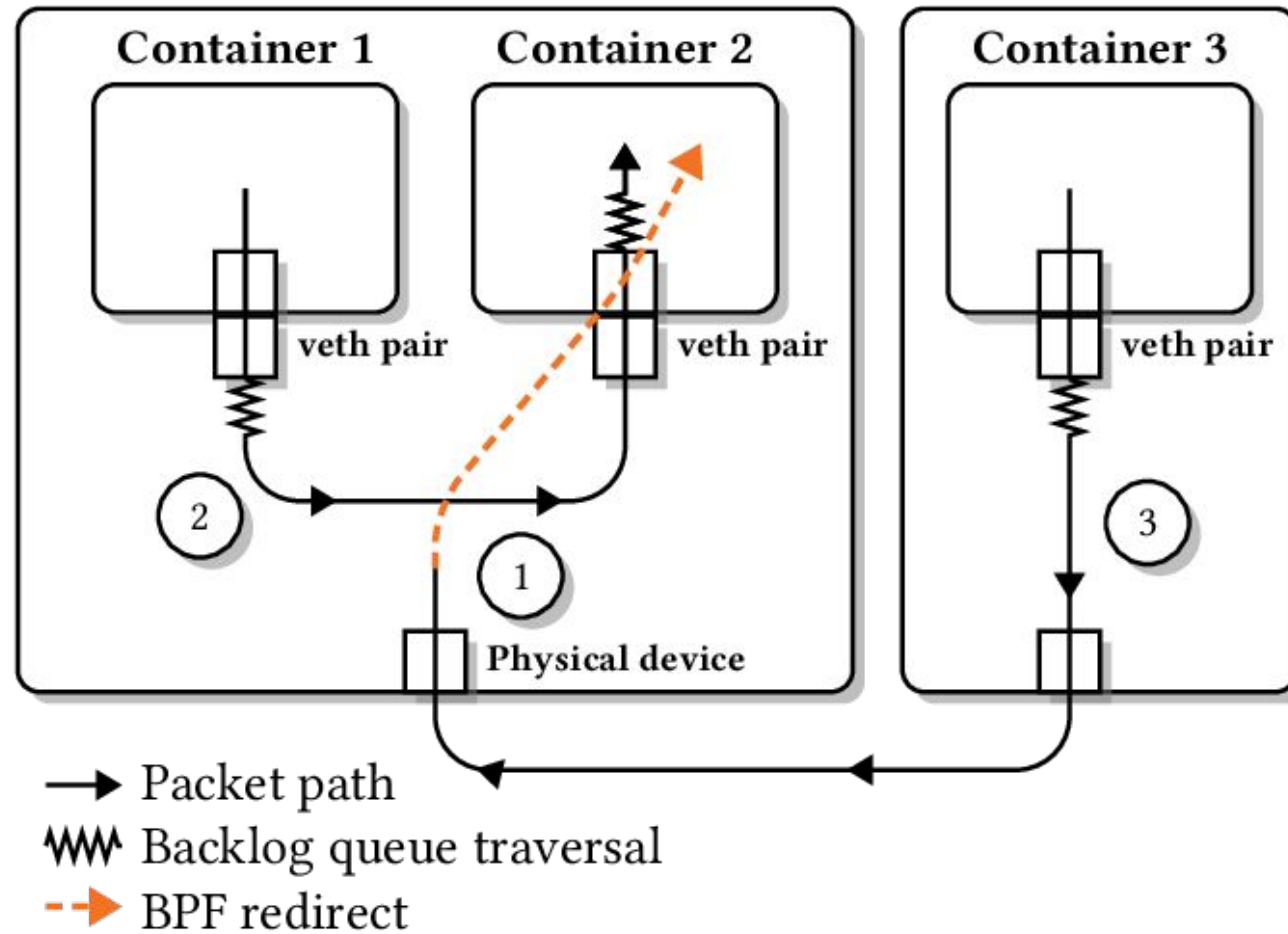
# netkit: Eliminating Backlog Queue Traversals



# Namespace-aware BPF redirect



# The netkit Device Pairs



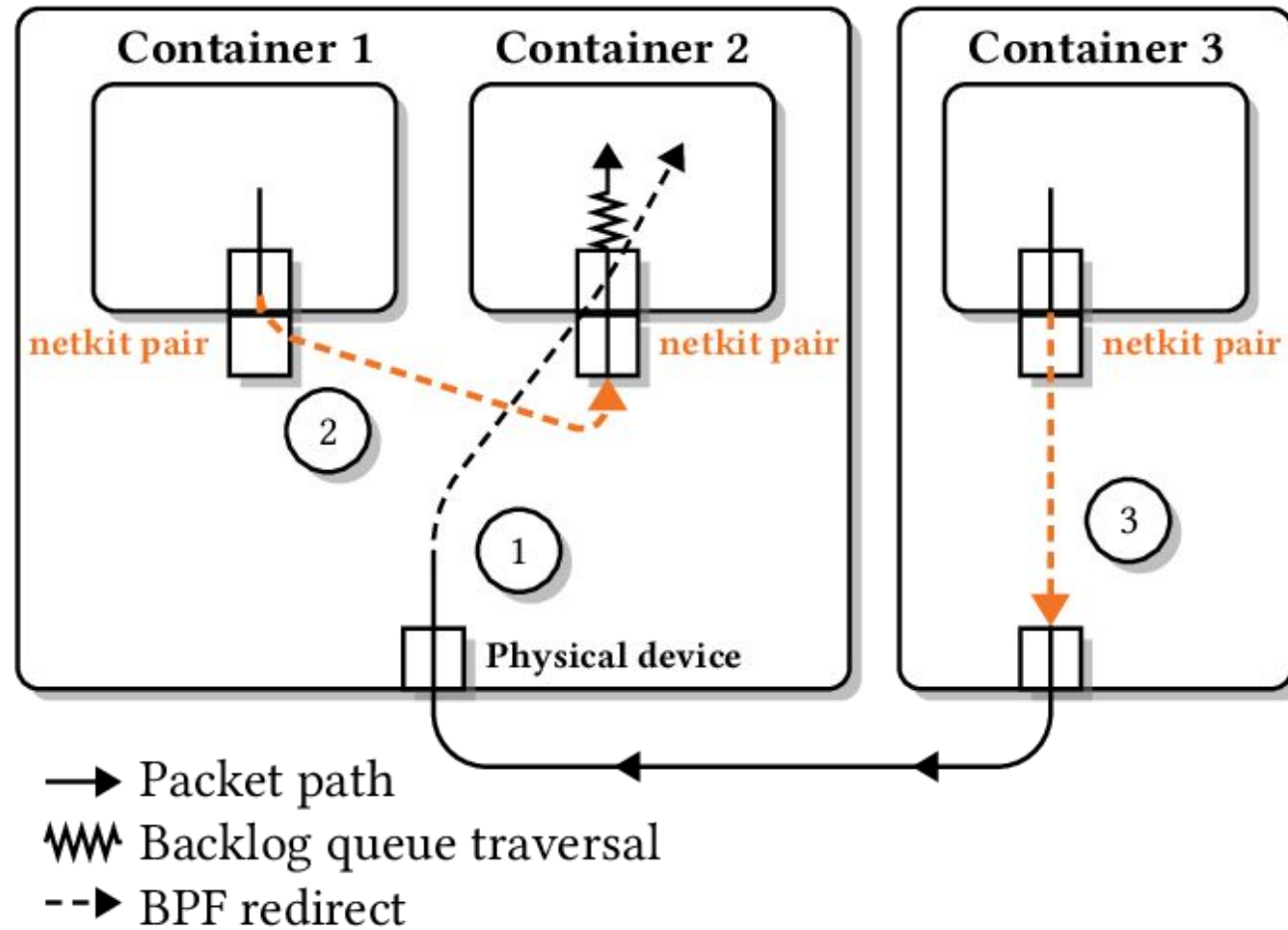
# The netkit Device Pairs



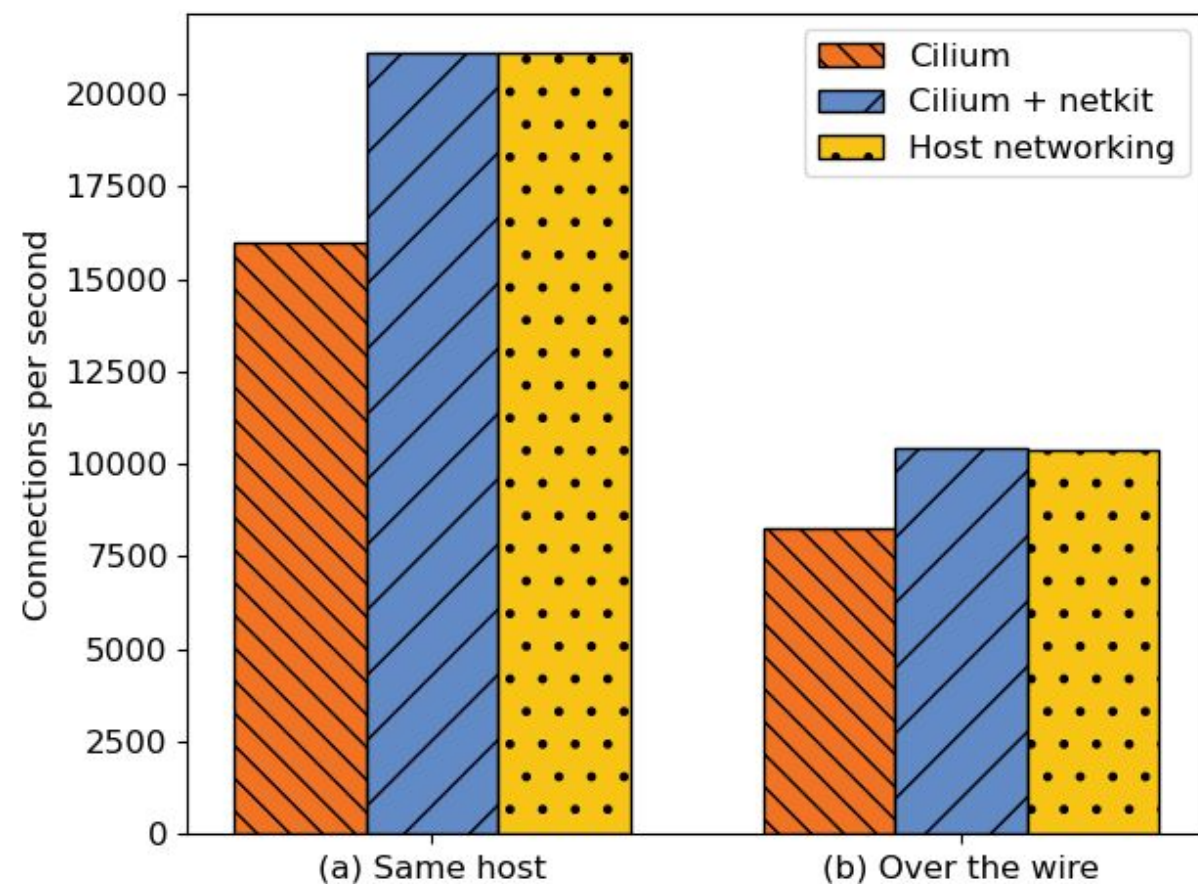
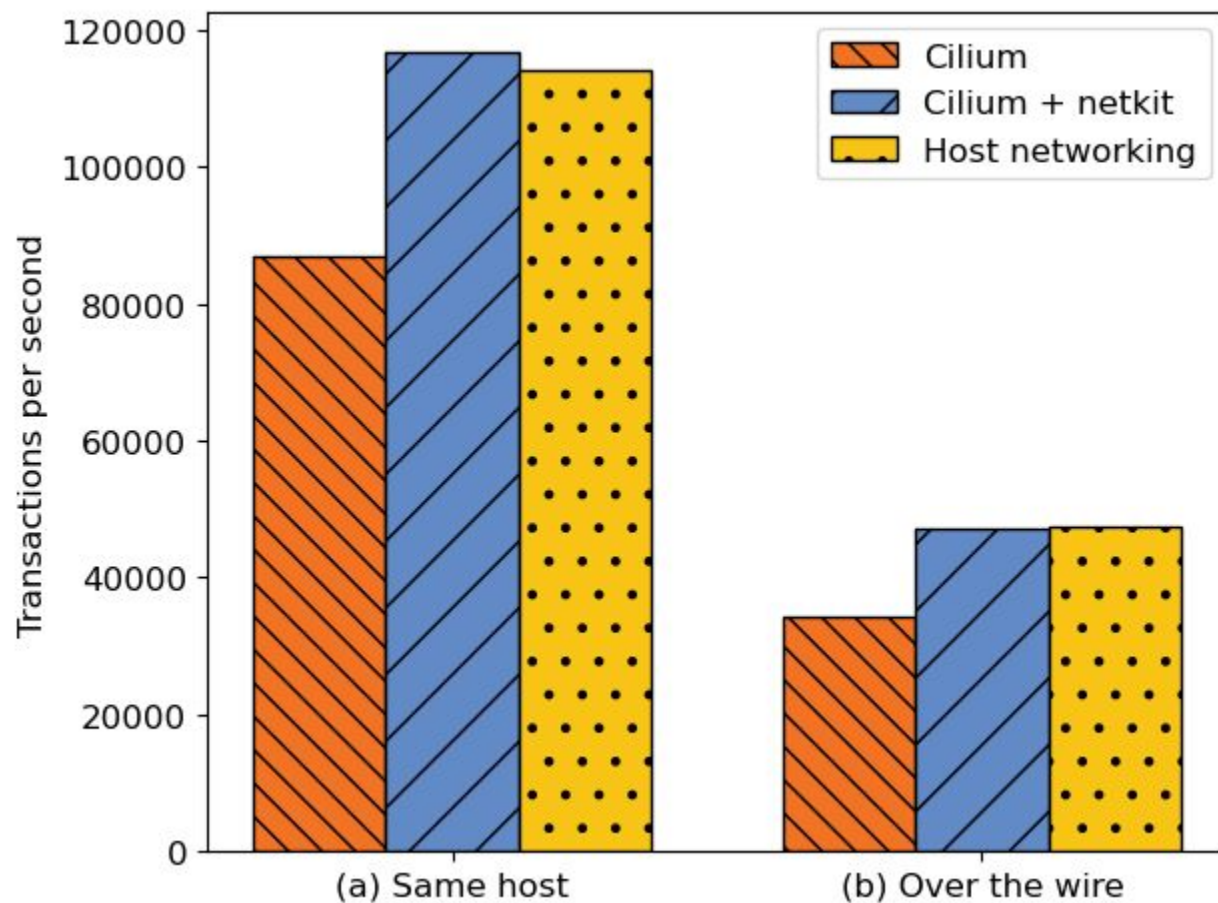
- “Container-aware” device pair: primary device in host, peer device in network namespace.
- Attached BPF programs executed on TX path, immediately after namespace switch.
- BPF programs for both sides attached through the primary device.

Result: 1) Applications can’t detach peer BPF programs.  
2) Peer BPF programs can access host information.

# The netkit Device Pairs



# Evaluations: TCP\_RR and TCP\_CRR

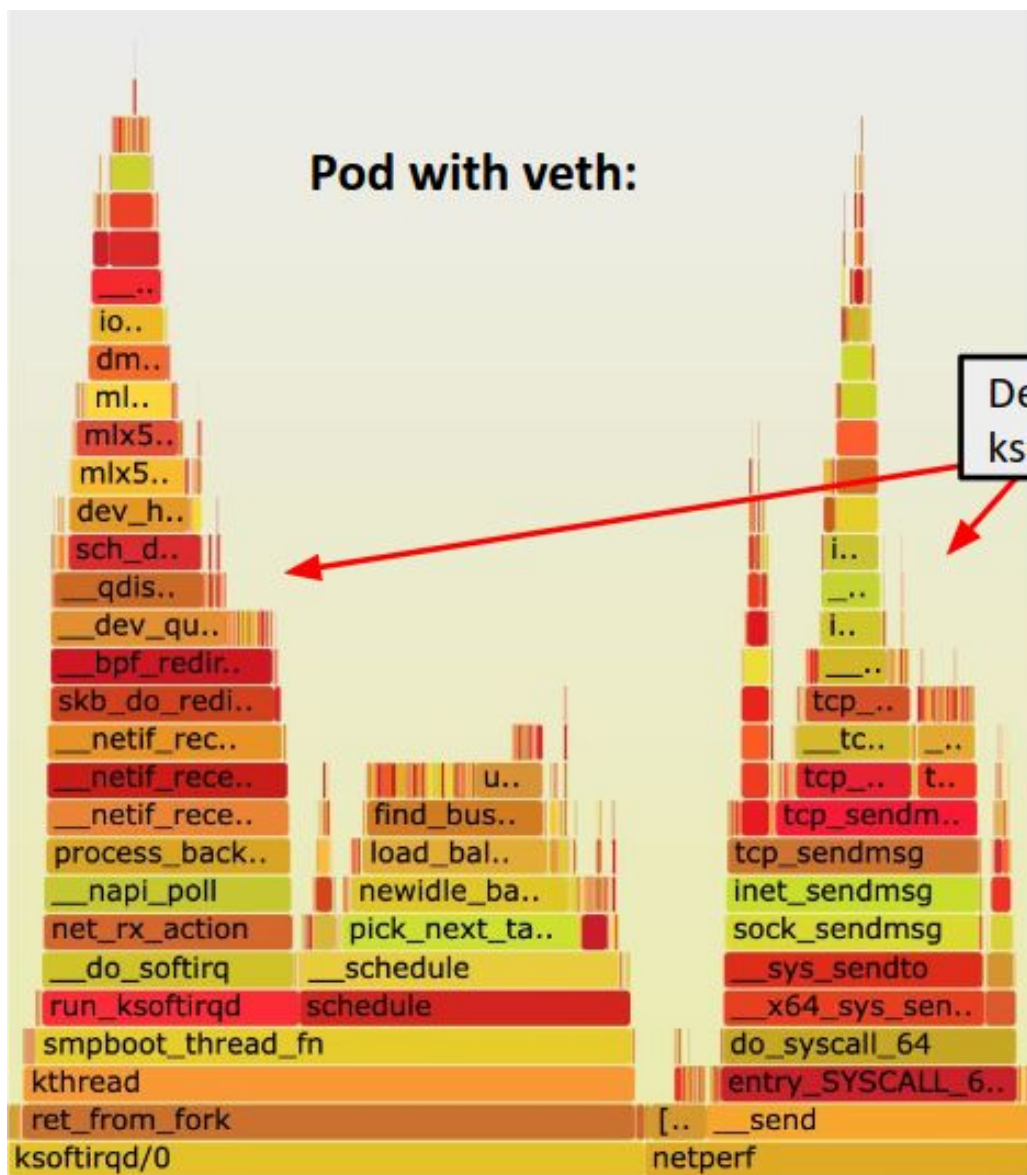


# Evaluations: CPU Accountability



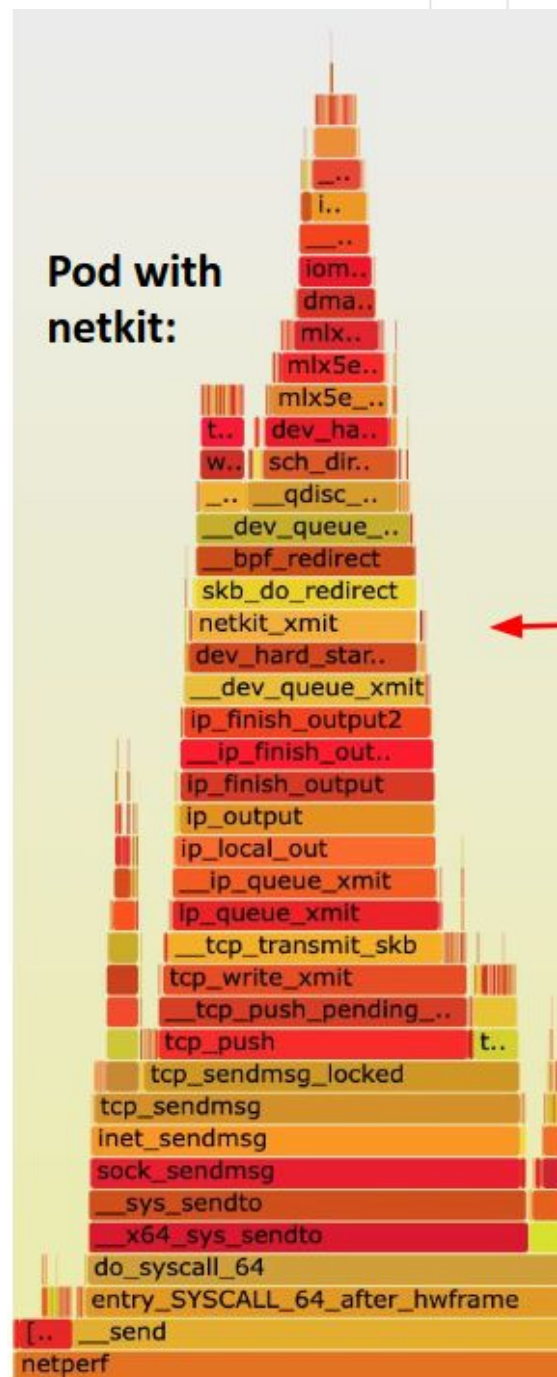
cilium

Pod with veth:



Deferral to  
ksoftirqd

Pod with  
netkit:



Remains in process  
context all the way,  
leading to better process  
scheduler decisions.

# Conclusion

- BPF to specialize the Linux network stack for containers
- netkit eliminates network namespace overhead
- Used in several large production container networks



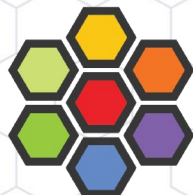
# Conclusion



- BPF to specialize the Linux network stack for containers
- netkit eliminates network namespace overhead
- Used in several large production container networks

Lots more to build:

- Accelerate RDMA (netdev 0x1A)
- Combined with AF\_XDP to speed up KubeVirt (Plumbers'25)



cilium

**Thank you!**

